

디지털포렌식

SafeTensors low-nibble steganography

2026년 7월

결과 요약

CRYPTO{G00D_J0B!_y0u_f0und_7h3_h1dd3n_s3cr37_1n_LLM}

1 최종 재현과 입력 무결성

제출 코드 solve.py는 Python 표준 라이브러리만을 사용하여 동작한다. 공식 TinyLlama ZIP 압축 파일에서 SafeTensors 형식의 대상 임베딩 행(embedding row)만을 순차적으로 읽어 F32 바이너리 데이터의 하위 니블(low nibble)을 병합한 뒤 플래그를 출력한다.

```
cd src
python3 solve.py TinyLlama-1.1B-Chat-v1.0.zip \
--verify-sha256
```

입력 파일명	공식 SHA-256 해시값
TinyLlama-1.1B-Chat-v1.0.zip	144155ad4b55ecf4e14f08457d4d8874ef656ea69e29632cc55bb97057269fa7
server.zip	1a0ecbdcb1ed4e3e51069643690659002612fbccc5a7a27919df17b7ab49dd5c
server.log (압축 해제본)	d7e3c1fb4c94754c80cedddb2a6caa0fb7f2aa6a3344bcb9a25e132d1f4cd5

--verify-sha256 옵션 실행 시, 모델 ZIP 파일을 8MiB 청크 단위로 스트리밍하여 전체 해시값을 검증한다. 이 과정에서 메모리에 전체 모델 또는 개별 텐서 데이터를 완전히 적재하지 않아 리소스 소모를 방지한다.

2 로그 분석에서 얻은 단서

17GB 크기의 server.log를 압축 파일에서 스트리밍 방식으로 한 줄씩 읽어 분석하였다. 각 chat 레코드의 q1, a1 필드는 base64 패딩(=)을 보충하여 디코딩한 후 카이사르 암호(Caesar Cipher, +11 shift)를 적용해 평문을 복원하였다. 복원된 평문 중 secret, square, flag, crypto, 4 bit, nibble 등의 키워드를 포함하는 레코드만을 필터링하였다.

식별된 핵심 레코드는 다음과 같다.

```
model: TinyLlama/TinyLlama-1.1B-Chat-v1.0
chat_id: chat-000005
Q: What secret does the model have?
A: The LLM knows the secret. Trust me.
"square" for open sesame!!!
```

복호화된 평문 끝의 'l' 문자는 패딩 여부가 불확실하므로 임의로 제거하지 않고 온전하게 보존하였다. 또한 square 키워드 질의에 대한 응답으로 기록된 "Wouldn't about 4 bits be enough?"는 가중치 데이터의 하위 4비트(low 4-bit) 영역을 분석하는 결정적인 가설이 되었다. 당시 generation 환경은 보존되지 않았으므로 이 응답의 결정론적 재현 가능 여부를 최종 공격의 전제로 삼지는 않았다.

3 F32 텐서 이상치 탐색 (Blind Discovery)

SafeTensors 형식 내의 F32 값은 little-endian 4바이트 구조를 가진다. 각 원소의 첫 번째 바이트에서 다음 식을 통해 하위 니블(low nibble)을 추출하였다.

$$nibble = raw_f32_bytes[0] \& 0x0f$$

모든 F32 텐서를 8MiB 청크 단위로 순차 처리하며 0이 아닌 하위 니블의 빈도와 복원된 ASCII 바이트열을 비교하였다. 그 결과 가장 유력한 후보 텐서와 행을 식별하였다.

```
tensor = model.embed_tokens.weight
row     = 0
non-zero low nibbles = 345
```

대용량 로그 파일의 무결성 증거 보존, 전체 텐서 탐색 및 예외 처리 로직은 분석용 코드(digital_forensics.py)로 분리하였으며, 제출용 코드(solve.py)에는 식별된 텐서 정보만을 반영하여 실행 단계를 최소화하고 가독성을 높였다.

분석용 코드는 스캔 중 후보 텐서가 발견되면 라인 번호, 바이트 오프셋, 라인/레코드 SHA-256, 매칭 키워드, 복호화 질문 및 답변 등을 포함한 무결성 보고서(log-evidence.jsonl)를 즉시 생성한다. 또한 최대 라인 길이를 제한하여 임의의 대형 로그 입력 상황에서도 메모리 점유율을 일정하게 관리한다.

4 페이로드 추출

SafeTensors의 data_offsets는 절대 오프셋이 아닌 JSON 헤더 종료 지점(data buffer 시작점) 기준의 상대 오프셋이다. 페이로드 추출은 다음 절차로 수행된다.

1. ZIP 압축 파일 내의 단일 .safetensors 파일을 확인한다.
2. 처음 8바이트를 읽어 JSON 메타데이터의 크기를 파싱한 뒤 헤더 내용을 분석한다.
3. model.embed_tokens.weight가 유효한 2차원 F32 텐서인지 확인한다.
4. 데이터 오프셋 정보를 토대로 첫 번째 행(row 0)만을 특정하여 읽어들인다.
5. 각 F32 원소의 첫 번째 바이트에서 하위 니블(low nibble)을 추출한다.
6. 연속된 두 니블을 $(high \ll 4) | low$ 수식으로 결합하여 1바이트 데이터로 복원한다.
7. 복원된 바이트열에서 정규식을 활용하여 플래그(CRYPTO{...})를 검출한다.

분석용 파서는 헤더 무결성(중복 키 검사, 정렬 오류 검증) 및 데이터 영역 겹침(overlap)이나 빈 공간(hole) 존재 여부를 정적 분석하며, 비정상적인 정렬 상태 발견 시 오류를 발생시키고 즉시 종료(fail-closed)한다.

추출 과정에서는 임의의 데이터 왜곡을 방지하기 위해 0 니블이나 0 바이트 값을 생략하지 않으며, 부동 소수점(float) 변환 과정 없이 원시 바이너리 수준에서 디코딩을 처리한다. 복원된 페이로드는 다음과 같다.

```
"The lighthouse opens at midnight. Meet me at the bar."
```

```
Well done, Agent H.
You traced the secBet into the model itself.
The flag is:
CRYPTO{G00D_J0B!_y0u_f0und_7h3_h1dd3n_s3cr37_1n_LLM}
```

secBet 및 open sesame의 오타는 원본 데이터 상의 텍스트를 그대로 보존한 결과이다. 최종 추출 코드는 복원된 바이트열에서 플래그 문자열이 실제로 매칭되어야 성공으로 판정한다.

5 합성 데이터 검증 및 재현 범위

```
cd src
python3 -m unittest -v test_digital_forensics
```

동작 검증을 위해 소형 합성 ZIP 파일을 활용한 회귀 테스트를 수행한다. 이는 복원 왕복(round-trip), 청크 및 니블 경계 처리, 비정상 구조(중복 키, 겹침, 미정렬) 예외 처리 등을 검증하여 파서 로직과 스트리밍 동작 방식의 무결성을 보장한다.

공식 TinyLlama ZIP 파일과 대용량 server.zip은 배포 전용 대형 입력 파일이므로 소스 트리 내에 중복 보존하지 않았다. 현재 검토 환경에서는 해당 공식 입력 파일이 부재하므로 본 보고서에 수록된 수치는 기존의 분석 결과물에 기재된 정보에 근거한다. 공식 파일이 제공될 경우, 제출된 검증 스크립트를 통해 SHA-256 무결성 및 플래그 복원이 결론론적으로 재현된다.

blind discovery 단계부터 재수행 및 제3자 검토를 하려면 공식 입력으로 생성한 log-evidence.jsonl 파일과 텐서 통계 데이터인 discovery.json 파일의 보존이 필요하다.

6 참고 자료

- [SafeTensors 공식 포맷 명세](#): 헤더 구조, 상대적 data_offsets, Little-endian Row-major 데이터 형식 및 검증 규칙.
- [TinyLlama-1.1B-Chat-v1.0 공식 모델 카드](#): 로그에 나타난 모델 식별자 정보.
- Zhang et al., *TinyLlama: An Open-Source Small Language Model*: 대상 모델군 분석 자료.